

UNITÉ 7 · SEMAINES 10–12

Threads et concurrence

📅 Semaines 10–12 · dates à confirmer </> Laboratoire TD6 (corrigé + défi IA) ~ 5 animations

MANUEL M7 · THÉORIE DE L'UNITÉ

Threads et concurrence

Lecture de base de l'unité 7 (≈ 2 h). Les sorties sont réelles: programmes du TD6 exécutés sur la machine du cours (28 cœurs x86-64).

SOMMAIRE

- | | | | |
|----|---|----|---|
| §1 | Threads | §4 | Mutex, sémaphores et variables de condition |
| §2 | Conditions de course et atomicité | | |
| §3 | Exclusion mutuelle: du logiciel au matériel | §5 | Interblocages |

§1 Threads

Un **thread** est une exécution séquentielle à l'intérieur d'un processus. Les threads d'un processus partagent tout l'espace d'adressage — code, données globales, tas — et les descripteurs; chacun a sa propre **pile**, ses **registres** et ses informations d'ordonnancement. Il n'y a pas de protection mémoire entre eux: le bogue d'un thread peut corrompre les données de tous, et un signal comme SIGSEGV termine tout le processus. Sous Linux, le noyau ordonnance chaque thread comme une tâche de plus (ils sont créés avec `clone`, en partageant la mémoire et les descripteurs).

L'API POSIX: `pthread_create(&t, attr, f, arg)` exécute `f(arg)` dans un nouveau thread; `pthread_exit` (ou le retour de `f`) le termine; `pthread_join` attend sa fin, récupère la valeur de retour et libère ses ressources (ou `pthread_detach`, si personne ne l'attendra). Ces fonctions renvoient 0 ou un numéro d'erreur, pas -1. Compilez avec `-pthread`. Créer un thread coûte beaucoup moins cher que créer un processus, car aucune table des pages n'est copiée:

Sortie réelle

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

Sortie réelle

```
$ ./threads
thread 57632 of process 357789: shared[0x650ff476602c] = 1, local[0x7626e31fee74] = 1
thread 79520 of process 357789: shared[0x650ff476602c] = 4, local[0x7626e19fbe74] = 1
thread 19968 of process 357789: shared[0x650ff476602c] = 5, local[0x7626dbffee74] = 1
thread 64928 of process 357789: shared[0x650ff476602c] = 2, local[0x7626e29fde74] = 1
thread 72224 of process 357789: shared[0x650ff476602c] = 3, local[0x7626e21fce74] = 1
thread 86816 of process 357789: shared[0x650ff476602c] = 6, local[0x7626e11fae74] = 1
thread 94112 of process 357789: shared[0x650ff476602c] = 7, local[0x7626e09f9e74] = 1
thread 27264 of process 357789: shared[0x650ff476602c] = 8, local[0x7626db7fde74] = 1
process 357789: all 8 threads finished, shared = 8
```

Pratiquez → A7.4 threads contre processus · TD6 · 1

§2 Conditions de course et atomicité

Une ligne de C comme `x = x + 2` représente plusieurs étapes du processeur : lire `x` dans un registre, additionner et écrire. Si deux threads exécutent de telles lignes sur la même variable, l'ordonnanceur peut entrelacer leurs étapes de n'importe quelle façon, et dans certains ordres une écriture **écrase** l'autre : c'est une **condition de course**. Pour `x = x+2 || x = x*3` avec `x = 1`, il y a 20 entrelacements possibles ; seuls les deux séquentiels donnent 9 ou 5, les autres perdent une mise à jour. En pratique, les « mauvais » ordres sont rares, ce qui rend ces bogues difficiles à reproduire : dans le TD6, huit threads qui vendent 100 000 billets sans protection en vendent quelques milliers de trop.

Une opération est **atomique** si elle s'exécute entièrement ou pas du tout, sans qu'un autre thread voie un état intermédiaire. Depuis C11, deux accès à la même variable depuis des threads différents, sans synchronisation et avec au moins une écriture, sont une *data race* et le comportement est indéfini ; `volatile` ne règle rien. Pour des opérations atomiques, utilisez `<stdatomic.h>` ou `__atomic_*`.

Sortie réelle

```
$ ./tickets_race 100000 | tail -3
[5] I sold 14026 tickets
[7] I sold 14753 tickets
103195 tickets sold out of 100000 (tickets = -7)
```

Pratiquez → A7.1 condition de course · TD6 · 2

§3 Exclusion mutuelle: du logiciel au matériel

Une **section critique** est un fragment qu'un seul thread à la fois peut exécuter (**exclusion mutuelle**). Le problème du « trop de lait » montre à quel point c'est difficile avec de simples lectures et écritures: regarder puis acheter laisse une fenêtre; une note partagée aussi (la regarder et la poser sont deux étapes); une note par thread évite d'acheter deux fois mais permet que personne n'achète. L'**algorithme de Peterson** (sa propre note + céder le tour + attendre tant que l'autre a une note et que c'est son tour) est correct si la mémoire est *séquentiellement cohérente*.

Les vrais processeurs ne le sont pas. Sur x86, chaque cœur laisse ses écritures dans un **tampon** et peut lire la mémoire avant qu'elles soient visibles pour les autres (le modèle TSO); ARM réordonne encore plus. Les deux threads de Peterson peuvent donc lire la note de l'autre à 0 et entrer ensemble. Mesuré sur la machine du cours, avec et sans barrière mémoire (`mfence`):

Sortie réelle

```
$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do ./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
```

C'est pourquoi l'exclusion mutuelle se construit avec des **instructions atomiques matérielles**: test-and-set, compare-and-swap (`lock cmpxchg` sur x86), fetch-and-add. Un verrou qui attend dans une boucle (*spinlock*) brûle du CPU et peut subir une *inversion de priorité*; la solution de Linux est le **futex**: si le verrou est libre, on le prend avec une instruction atomique en espace utilisateur, et on n'appelle le noyau pour dormir que s'il faut attendre. Sans contention, un mutex ne fait pratiquement aucun appel système:

Sortie réelle

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e trace=futex
./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
  0.00    0.000000          0          1          futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00    0.223730         27        8013        3116 futex
```

Pratiquez → A7.5 le problème du lait · TD6 · 5

§4 Mutex, sémaphores et variables de condition

Un **mutex** (`pthread_mutex_lock` / `unlock`) protège une section critique: celui qui trouve le mutex pris dort jusqu'à ce qu'il soit libéré. Un **sémaphore** (Dijkstra) est un entier ≥ 0 avec deux opérations atomiques: `sem_wait` (P: attend qu'il soit positif et retire 1) et `sem_post` (V: ajoute 1 et réveille un thread en attente). Initialisé à 1, c'est un verrou; à 0, une notification entre threads; à k, une limite de k utilisateurs à la fois. `sem_init` en crée un anonyme (partageable entre processus avec `pshared = 1` en mémoire partagée) et `sem_open` un nommé (il renvoie `SEM_FAILED` en cas d'échec).

Le **producteur-consommateur** combine les deux: un tampon de `BUF` cases, un sémaphore `empty` (cases libres, qui commence à `BUF`), un `full` (cases occupées, qui commence à 0) et un mutex pour les indices partagés. Le producteur attend si le tampon est plein et le consommateur s'il est vide; rien ne se perd ni ne se répète:

Sortie réelle

```
$ ./prodcons_multi 4 100000
consumer 2: 23918 items
consumer 3: 26423 items
consumer 1: 23037 items
consumer 0: 26622 items
sum = 5000050000, expected 5000050000: OK
```

Pour attendre une condition quelconque, on utilise des **variables de condition**: mutex pris, `while (!condition) pthread_cond_wait(&c, &m);`. `cond_wait` relâche le mutex et dort de façon atomique, puis le reprend au réveil; celui qui change la condition appelle `pthread_cond_signal` ou `_broadcast`. On utilise `while` parce qu'il peut y avoir des réveils intempestifs.

Pratiquez → A7.2 producteur-consommateur · TD6 · 3

§5 Interblocages

Un **interblocage** est un ensemble de threads qui s'attendent mutuellement pour toujours. Il n'apparaît que si **les quatre conditions** (Coffman) sont réunies: exclusion mutuelle, détention de ressources en attendant d'autres, pas de réquisition et attente circulaire. Comme il dépend de l'ordre d'exécution, il peut ne pas apparaître en des milliers de tests. Le dîner des philosophes le montre: si chacun prend d'abord la baguette de droite puis celle de gauche, tous peuvent finir par en tenir une et attendre celle du voisin. Il suffit de rompre une condition: un **ordre global** d'acquisition (la correction du TD6), limiter le nombre de concurrents (un « serveur »), relâcher et réessayer (`trylock`), ou détecter les cycles dans le graphe d'allocation et récupérer. Dans le noyau Linux, `lockdep` vérifie que les verrous sont toujours pris dans le même ordre.

Sortie réelle

```
$ timeout 3 ./philosophers_deadlock 3 | tail -8; echo "exit status: ${PIPESTATUS[0]}"
(124 = still blocked after 3 s)"
7: hungry
2: hungry
3: hungry
1: hungry
0: hungry
6: hungry
4: hungry
5: hungry
exit status: 124 (124 = still blocked after 3 s)
```

Sortie réelle

```
$ ./philosophers 3 | tail -3
7: full
7: done after 3 meals
dinner finished: 8 philosophers x 3 meals
```

Pratiquez → A7.3 les philosophes · TD6 · 4